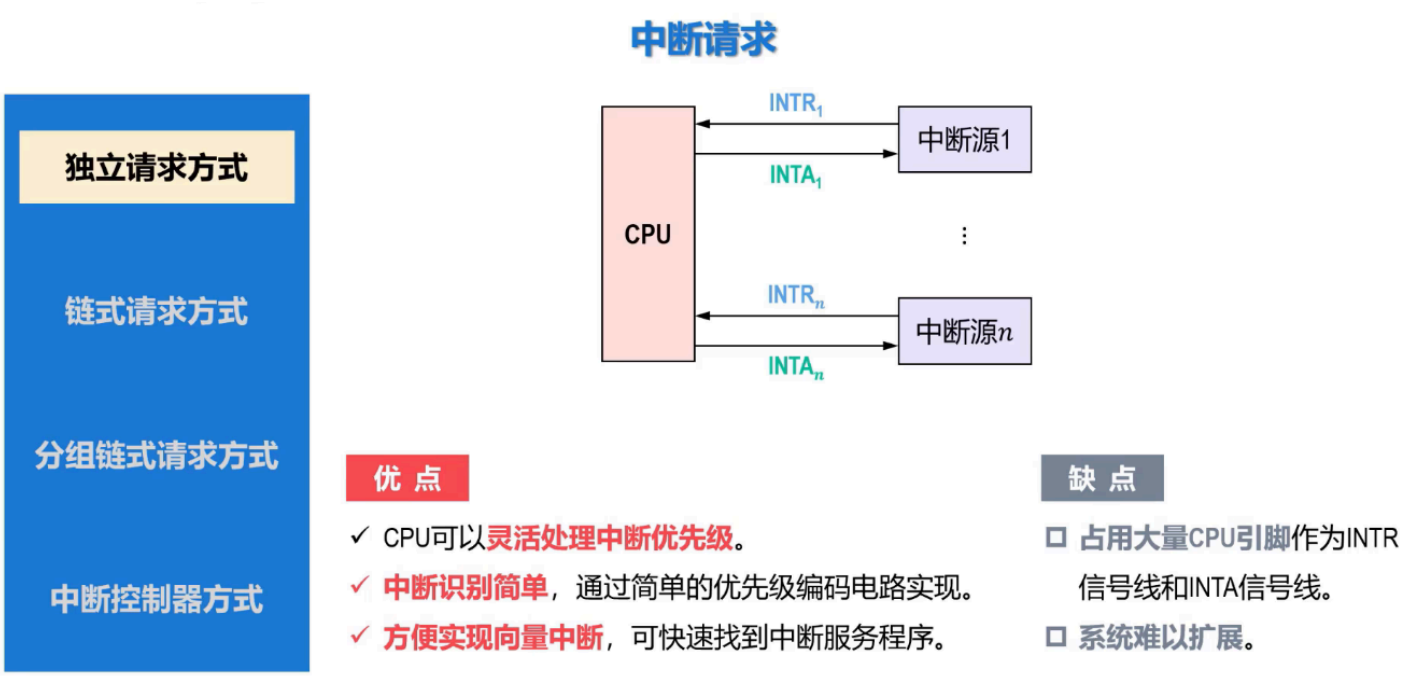
8.9 中断请求

考纲定位：计算机组成原理 · 第 8 章 I/O 系统（8.9）｜重要度 ★★★★｜常见考法：选择题（四种请求方式、8259A 内部寄存器、INTR/INTA/EOI 时序），偶作流程图分析

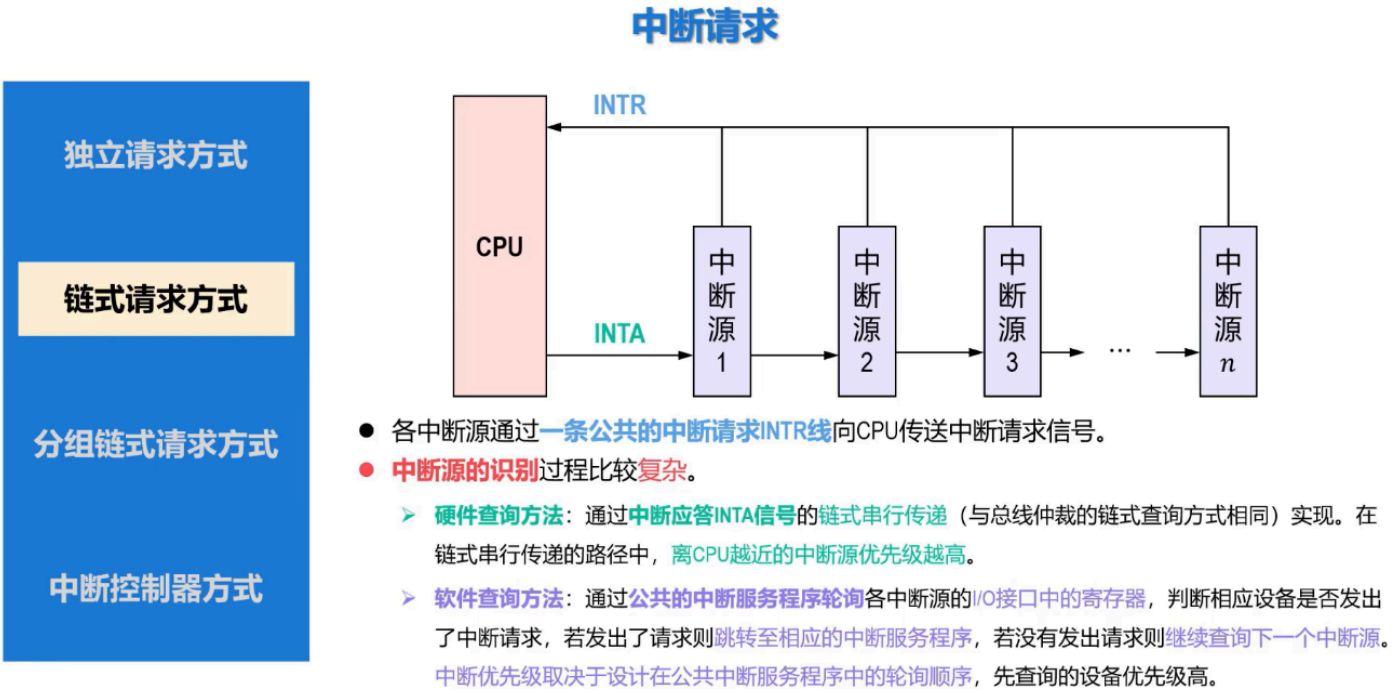
一句话核心：中断请求解决“设备如何告诉 CPU”和“CPU 如何识别中断源”；四种方式在引脚数、扩展性、响应速度之间权衡

一、独立请求方式



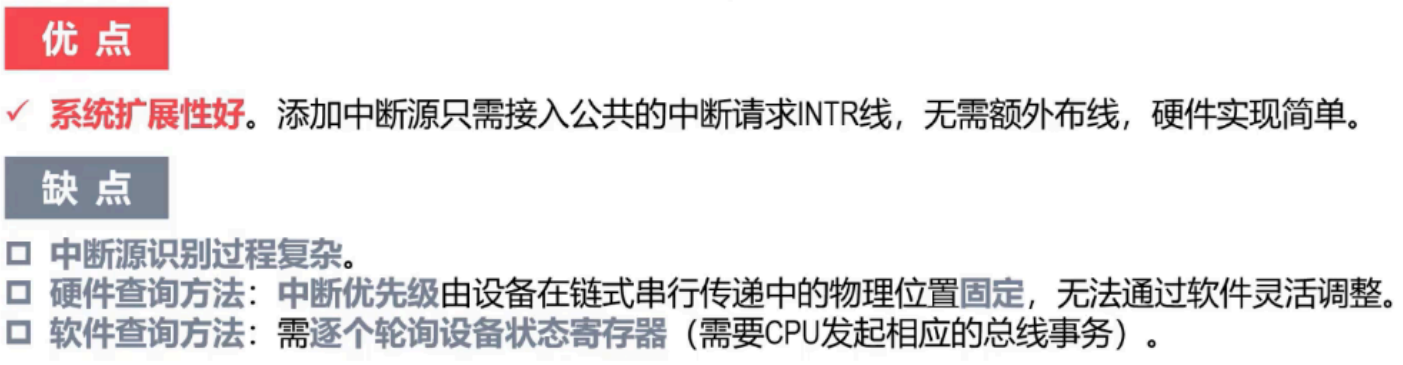
- 每个中断源都有独立的 INTR_i 和 INTA_i 信号线，直接连到 CPU
- 优点
  - CPU 可以灵活处理中断优先级
  - 中断识别简单，通过简单的优先级编码电路实现
  - 方便实现向量中断，可快速找到中断服务程序
- 缺点
  - 占用大量 CPU 引脚作为 INTR/INTA 信号线
  - 系统难以扩展

二、链式请求方式



- 各中断源通过一条公共的中断请求 INTR 线向 CPU 传送中断请求信号
- CPU 发出 INTA 响应后，需要识别是哪个中断源
- 中断源识别有两种方法
  - 硬件查询法：INTA 信号沿链式串行传递（与总线仲裁的链式查询方式相同）；离 CPU 越近的中断源优先级越高
  - 软件查询法：公共的中断服务程序轮流查询各中断源的 I/O 接口中的状态寄存器，判断是谁发出请求；查询顺序决定优先级

优点



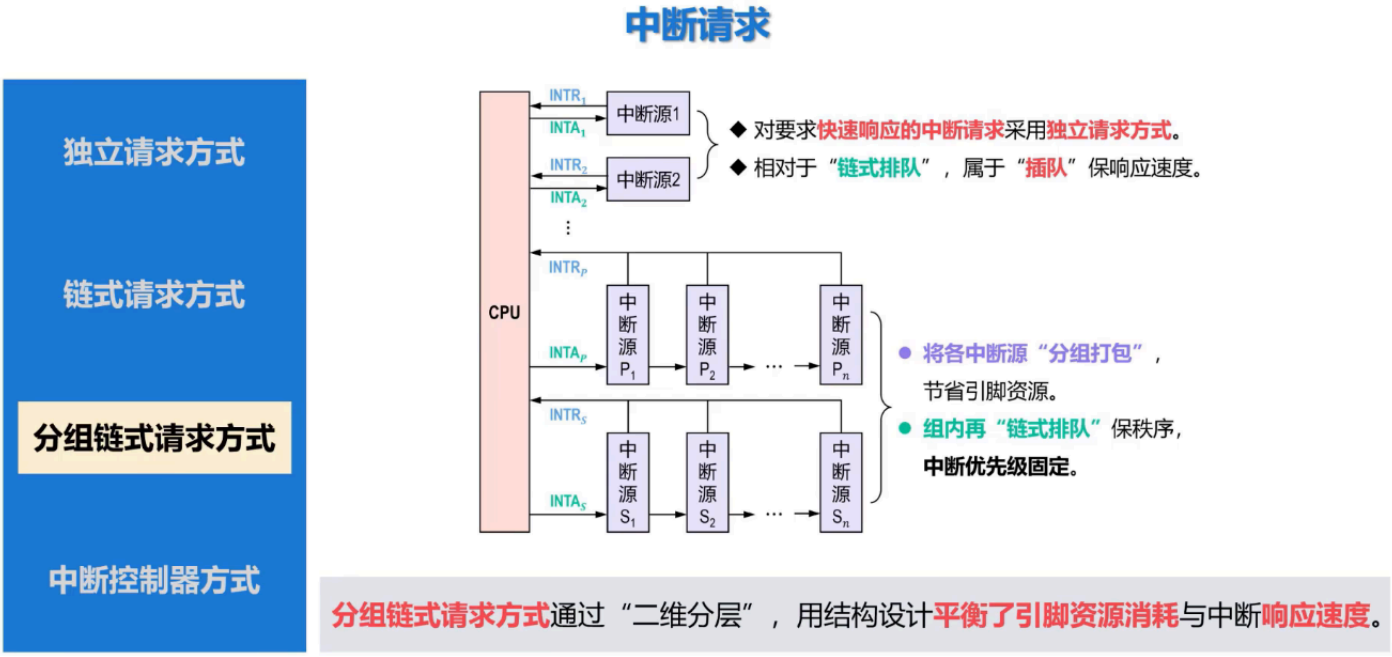
- 系统扩展性好：添加中断源只需接入公共 INTR 线，无需额外布线，硬件实现简单

缺点

- 中断源识别过程复杂
- 硬件查询法：中断优先级由设备在链式串行传递中的物理位置固定，无法通过软件灵活调整
- 软件查询法：需逐个轮询设备状态寄存器，需要 CPU 发起相应的总线事务

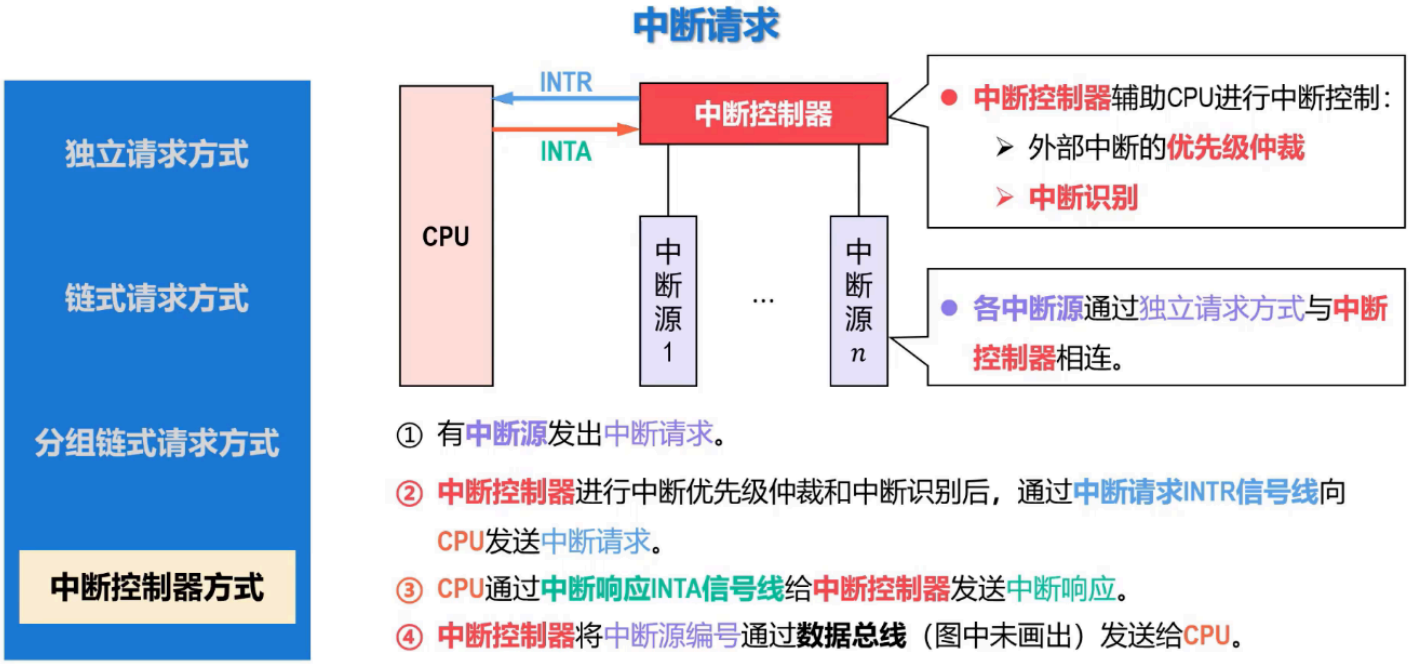
类似：7.5 总线仲裁

三、分组链式请求方式



- 把独立请求与链式请求结合：对要求快速响应的中断请求采用独立请求方式，相当于“插队”，保障响应速度
- 其余中断源分组，组内再按链式排队，节省引脚资源
- 组内按链式排队，中断优先级固定
- 通过“二维分层”用结构设计平衡引脚资源消耗与中断响应速度

四、中断控制器方式

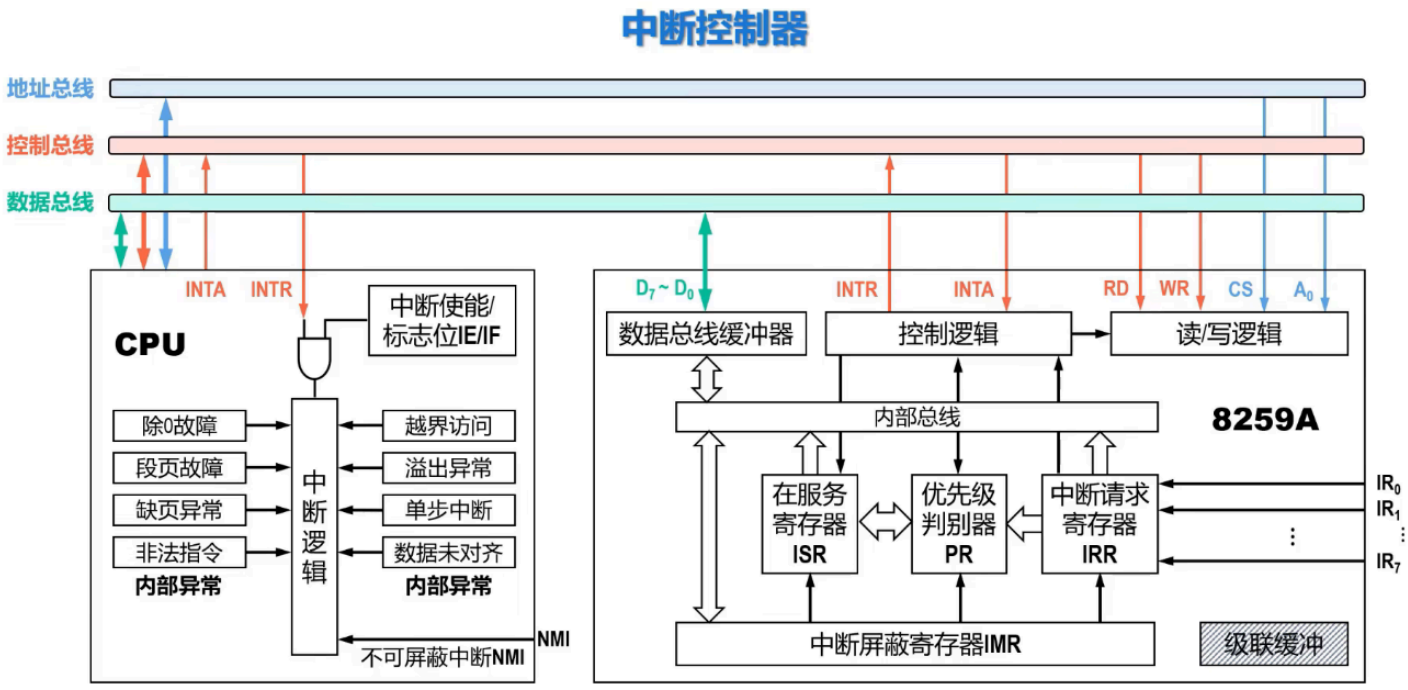


- CPU 通过中断控制器（如 8259A）管理中断
- 各中断源通过独立请求方式与中断控制器相连
- 中断控制器辅助 CPU 进行中断控制

- 外部中断的优先级仲裁
- 中断识别

- 基本流程
  1. 有中断源发出中断请求
  2. 中断控制器进行中断优先级仲裁和中断识别，然后通过 INTR 向 CPU 发中断请求
  3. CPU 通过 INTA 给中断控制器发中断响应
  4. 中断控制器将中断源编号通过数据总线（图中未画出）发给 CPU

4.1 中断控制器硬件逻辑图



CPU 侧中断逻辑：内中断与外中断的分流

分类维度	内中断（异常）	外中断（中断）
触发来源	CPU 执行指令内部产生	CPU 外部设备发出
是否与指令相关	同步，与当前指令绑定	异步，与指令无关
是否可屏蔽	一律不可屏蔽	INTR 可屏蔽（受 IF 位控制），NMI 不可屏蔽
典型例子	除 0、非法指令、溢出、缺页、单步、越界、数据未对齐	键盘/打印机等 I/O 中断、时钟中断、掉电（NMI）
检测时机	指令执行过程中检测	每条指令执行完毕的边界检测

- 图里 CPU 左侧的除 0、段页故障、缺页、非法指令、越界、溢出、单步、数据未对齐全部汇入“中断逻辑”，对应内中断（异常）
- 右侧经 NMI 直接进入的是不可屏蔽外中断（如掉电、硬件故障），不经过 8259A

- 经 INTR 进入的才是 8259A 送来的可屏蔽外中断，且要经过 IF 位的与门过滤
- IF=0 时，所有 INTR 信号都被挡掉；但 NMI 和内中断不受 IF 影响

8259A 内部核心部件

8259A 是 CPU 与外设之间的“中断管理者”，单片管理 8 级中断，通过级联最多扩展到 64 级

部件	全称	作用	关键特征
IRR	中断请求寄存器	锁存 IR0~IR7 上到达的中断请求信号	对应位置 1；被响应后该位才清 0
IMR	中断屏蔽寄存器	过滤 IRR 中的请求，被屏蔽位对应的中断不能进 PR	由 CPU 通过 OCW1 设置，每位独立屏蔽
PR	优先级判别器	从未被屏蔽的 IRR 请求中选优先级最高者，并与 ISR 中正在服务的中断比较	IR0 优先级最高，IR7 最低；支持嵌套判优
ISR	在服务寄存器	记录当前 CPU 正在服务的中断	响应后由 INTA 置位，EOI 后清 0

信号流顺序：设备请求 → IRR 锁存 → IMR 过滤 → PR 判优 → 通过 INT 向 CPU 的 INTR 发请求 → CPU 回 INTA → ISR 对应位置 1、IRR 对应位清 0 → 第二个 INTA 期间 8259A 把中断类型号放到 D7~D0 数据总线上 → CPU 据此查中断向量表跳转 ISR

完整的可屏蔽中断响应流程

① 中断请求与判优（8259A 内部）

设备在 IRn 上发请求 → IRR 置位 → IMR 判断是否屏蔽 → PR 比较 IRR 中未屏蔽请求与 ISR 中正在服务的中断的优先级 → 若更高，向 CPU 发 INT 信号

② 中断响应（硬件自动完成，即中断隐指令）

CPU 在一条指令执行完毕且 IF=1 时响应，硬件自动完成三件事：

- 关中断（清 IF，防止保护现场被打断）
- 保存断点（压 PC + PSW 入栈）
- 取向量、跳转 ISR 入口（从数据总线 D7~D0 取回类型号 → 查中断向量表 → 入口地址送 PC）

③ 中断服务（软件完成，ISR 内部）

保护现场（压通用寄存器） → 必要时开中断允许嵌套 → 执行具体处理 → 发 EOI 命令给 8259A（清 ISR 对应位） → 关中断 → 恢复现场 → 执行 IRET 中断返回

④ 中断返回

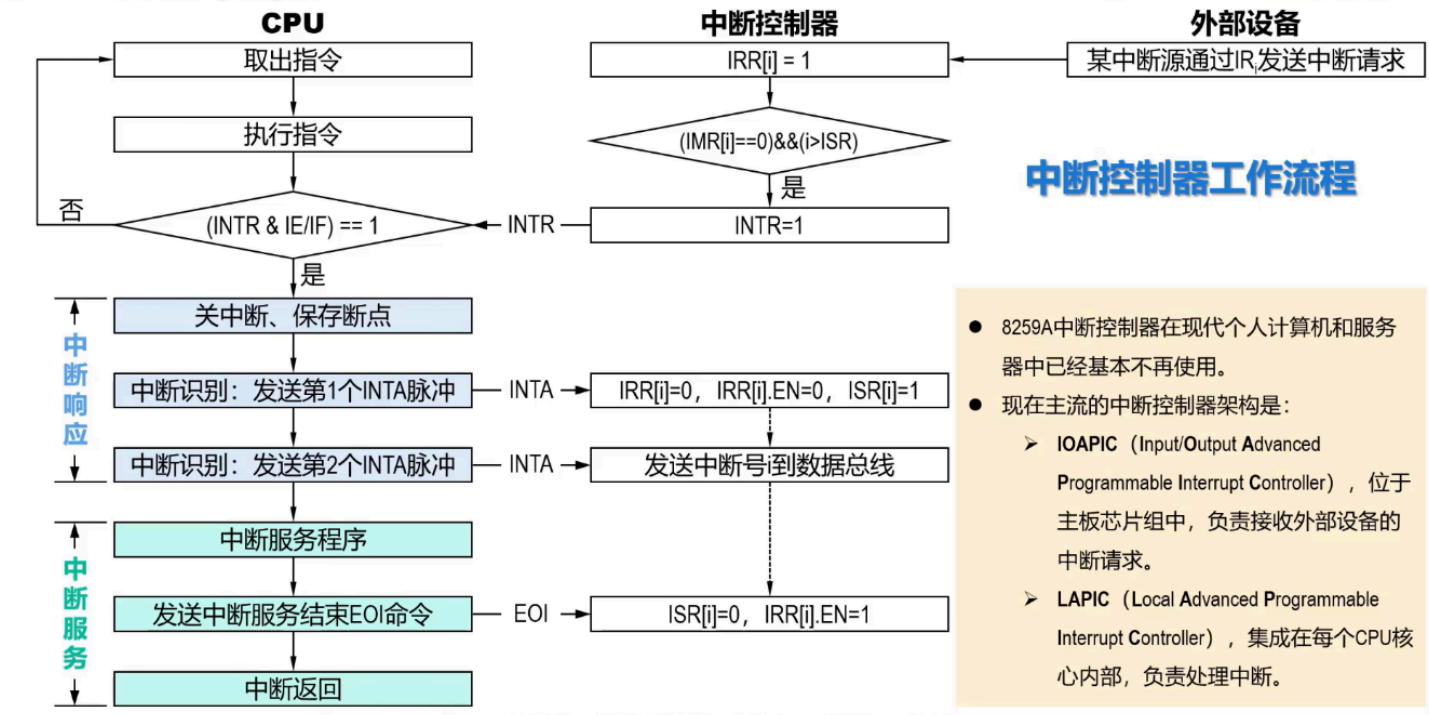
IRET 硬件自动弹出断点和 PSW，回到被打断的程序

408 高频易错点

- **保存断点与保护现场的执行主体不同**：保存断点（PC + PSW）是硬件在中断隐指令阶段自动完成；保护现场（通用寄存器）是软件在 ISR 里手动压栈完成

- **中断向量与向量地址不同**：中断向量是中断服务程序的入口地址本身；向量地址是中断向量表中存放该入口地址的内存单元地址
- **IF 位只对 INTR 有效**：IF=0 屏蔽所有经 8259A/INTR 来的可屏蔽中断，但 NMI 和所有内中断不受 IF 位控制
- **INTA 是两次脉冲**：第一个 INTA 通知 8259A“请求被接受”（ISR 置位、IRR 清零），第二个 INTA 期间 8259A 才把中断类型号放到数据总线上
- **中断响应的三个必要条件**：有未被屏蔽的中断请求、CPU 开中断（IF=1）、一条指令执行完毕
- **优先级判别是“双向比较”**：PR 不仅要在多个新请求中排队，还要把新请求与 ISR 中正在服务的中断比较，只有新请求优先级更高才会再次向 CPU 发 INT——这是中断嵌套的硬件基础
- **EOI 的时机影响嵌套**：如果 ISR 处理完没发 EOI，ISR 对应位一直保持 1，PR 会认为该中断还在服务，后续同级或更低级的中断无法被响应
- **8259A 规模**：单片管理 8 级中断，级联最多 64 级

4.2 中断控制器工作流程图



按“设备发请求 → 8259A 把关 → CPU 响应 → 执行服务 → 收尾返回”五幕理解，四根跨线信号（INTR、INTA、INTA、EOI）是幕与幕之间的交接棒

第一幕：请求诞生

- 外部设备（如键盘）在中断请求线 IRi 上拉高电平
- 8259A 把中断请求寄存器的对应位置 1，即 IRR[i] = 1
- 此时请求只是被 8259A 登记，CPU 完全不知道；设备不会直接敲 CPU 的门

第二幕：8259A 的层层把关

8259A 持续对 IRR 里的请求做两个判断，对应菱形 `(IMR[i]==0) && (i>ISR)`：

1. 过 IMR 这一关：IMR[i]==0 表示该中断源没有被屏蔽；如果 CPU 之前通过 OCW1 把这位屏蔽了，请求就死在这里

2. 过 PR 这一关：i>ISR 表示请求 i 的优先级高于 ISR 中正在服务的中断；新请求只有“打赢”正在服务的那一个，才值得去打扰 CPU

两关都过了，8259A 才在 INTR 线上置 1，向 CPU 发出正式申请

第三幕：CPU 的日常循环与响应时机

CPU 的永恒循环：取出指令 → 执行指令 → 检查 `(INTR & IE/IF) == 1` → 否，回到取出指令

- INTR：外面真的有人申请吗
- IE/IF：CPU 自己开中断了吗；IF=0 时，即使 INTR=1 也视而不见
- 检查时机在一条指令执行完的边界，CPU 绝不会在指令执行到一半时被可屏蔽中断打断
- 条件满足后进入中断响应阶段，这一段是硬件自动完成的，即中断隐指令

第四幕：中断响应——两次 INTA 的握手

1. 关中断、保存断点：CPU 先清 IF，然后把断点（PC + PSW）压栈
2. 发第 1 个 INTA 脉冲：CPU 对 8259A 说“你的申请我批准了”；8259A 做三件内部状态更新
    - IRR[i]=0：撤销挂起的请求
    - ISR[i]=1：登记“正在服务中”，成为 PR 判优的依据
    - IRR[i].EN=0：封锁该中断源，服务期间不允许它再次申请
3. 发第 2 个 INTA 脉冲：CPU 问“你是谁”；8259A 把中断类型号放上数据总线，CPU 取回后查中断向量表，得到 ISR 入口地址送入 PC

INTA 必须发两次脉冲：第一次是“确认 + 状态切换”，第二次才是“交付类型号”

第五幕：中断服务与收尾

- 控制权交给软件，CPU 执行中断服务程序：保护现场（压通用寄存器），必要时开中断（允许更高优先级请求嵌套——硬件基础是 PR 对 ISR 的比较），然后处理具体任务
- 收尾两件事，顺序不能乱
    1. 发送 EOI 命令：8259A 收到后 ISR[i]=0，IRR[i].EN=1；如果忘了发 EOI，ISR[i] 永远是 1，PR 会一直认为这个中断还在服务，同级和更低级的中断从此石沉大海
    2. 中断返回：IRET 自动弹回断点和 PSW，控制权回到主程序

四根跨线信号小结

信号	方向	时机	一句话含义
INTR	8259A → CPU	过了 IMR + PR 两关后	“有人申请，且优先级够格”
INTA 第 1 脉冲	CPU → 8259A	关中断保存断点之后	“批准了”→8259A 清 IRR、置 ISR
INTA 第 2 脉冲	CPU ← 8259A（经数据总线）	紧接第 1 脉冲	“你是谁”→送中断类型号
EOI	CPU → 8259A	服务程序末尾	“干完了”→清 ISR、解封中断源

延伸理解

- **中断嵌套**：图中 `(i>ISR)` 的判断出现在 INTR 发出之前，嵌套的准入门槛由 8259A 把守；CPU 侧能否真正嵌套，还要看 ISR 里是否及时开了中断。软硬件两侧条件同时满足，嵌套才发生
- **现代替代**：8259A 在现代计算机中已基本退役，取而代之的是 IOAPIC（位于芯片组，收外部设备中断）+ LAPIC（每个 CPU 核一个，处理中断）的两级架构；408 仍以 8259A 为教学模型，因为它把判优、屏蔽、嵌套、向量这些中断系统的本质机制展现得最完整

4.3 四种请求方式对比

方式	CPU 引脚数	优先级灵活性	识别速度	扩展性	典型场景
独立请求	多	高	快	差	引脚充足、少量高速设备
链式请求	少	低（硬件链固定或软件查询）	慢	好	设备多、速度要求不高
分组链式	中	组内固定、组间灵活	中	较好	平衡引脚与速度
中断控制器	少	高（可编程）	快	好（可级联）	现代计算机

五、易错辨析

四种请求方式

易混点	核心区分
独立请求 vs 链式请求	独立请求每个中断源有独立 INTR/INTA 线，快但引脚多；链式请求共用 INTR，省引脚但识别慢
链式硬件查询 vs 软件查询	硬件查询靠 INTA 链传播，优先级由物理位置固定；软件查询靠 ISR 轮询状态寄存器，优先级由查询顺序决定
分组链式 vs 纯链式	分组链式把“快设备”独立请求、“慢设备”组内链式，平衡引脚与速度
中断控制器 vs 独立请求	中断控制器把独立请求线收拢到控制器，CPU 只留 INTR/INTA，兼顾速度与扩展性

8259A 内部

易混点	核心区分
IRR / ISR / IMR	IRR 锁存请求；ISR 记录正在服务；IMR 是按位屏蔽
INTR / INTA	INTR 是 8259A 向 CPU 请求；INTA 是 CPU 向 8259A 响应

易混点	核心区分
两次 INTA	第 1 次确认并更新 IRR/ISR；第 2 次送中断类型号
EOI 的作用	清 ISR 对应位，解封中断源；漏发会导致后续同级/低级中断无法响应
中断类型号 vs 中断向量	类型号是 8259A 送出的编号；中断向量是 ISR 入口地址；向量地址是向量表中存放入口地址的单元地址
单片 8 级 vs 级联 64 级	8259A 单片 8 级，级联最多 64 级

六、考法透视

常见题型

- 给四种请求方式的图，判断引脚数、优先级灵活性、识别速度、扩展性
- 给 8259A 内部结构图，指出 IRR/IMR/ISR/PR 的作用
- 给中断流程，判断两次 INTA 各自完成什么
- 判断 EOI 漏发/时机错误会导致什么
- 结合 8.10/8.11，判断中断隐指令与 ISR 的分界

常见设错点

1. 认为链式请求的优先级可以通过软件灵活调整（硬件查询法不能）
2. 认为 INTA 只发一次
3. 认为第一个 INTA 就送中断类型号
4. 认为 EOI 由硬件自动发送
5. 认为 IF=0 能屏蔽 NMI
6. 把 IRR 和 ISR 的作用记反
7. 认为 8259A 单片管理 64 级

七、知识关联

- 前置：8.7 中断的概念、8.8 中断优先级和中断屏蔽
- 后续：8.10 单级中断处理流程（中断隐指令与 ISR）、8.11 多重中断的处理流程（开关中断与 EOI）
- OS 视角：中断控制器、中断向量、驱动中的 ISR
- 全章导航：8.0 导航与术语

八、费曼自测

1. 四种中断请求方式各自的核心权衡是什么 → 独立请求快但引脚多；链式请求省引脚但识别慢；分组链式折中；中断控制器用控制器收拢请求，兼顾速度与扩展
2. 链式请求的硬件查询和软件查询有什么区别 → 硬件查询靠 INTA 链传播，优先级由物理位置固定；软件查询由 ISR 轮询状态寄存器，优先级由查询顺序决定

3. 8259A 的 IRR、IMR、ISR、PR 各自负责什么 → IRR 锁存请求；IMR 按位屏蔽；ISR 记录正在服务；PR 判优并决定是否向 CPU 发 INT
4. 两次 INTA 分别完成什么 → 第 1 次确认请求，IRR 清零、ISR 置位；第 2 次把中断类型号放上数据总线
5. 如果 ISR 结束时忘记发 EOI 会怎样 → ISR 对应位一直为 1，PR 认为该中断仍在服务，同级和更低级中断无法被响应